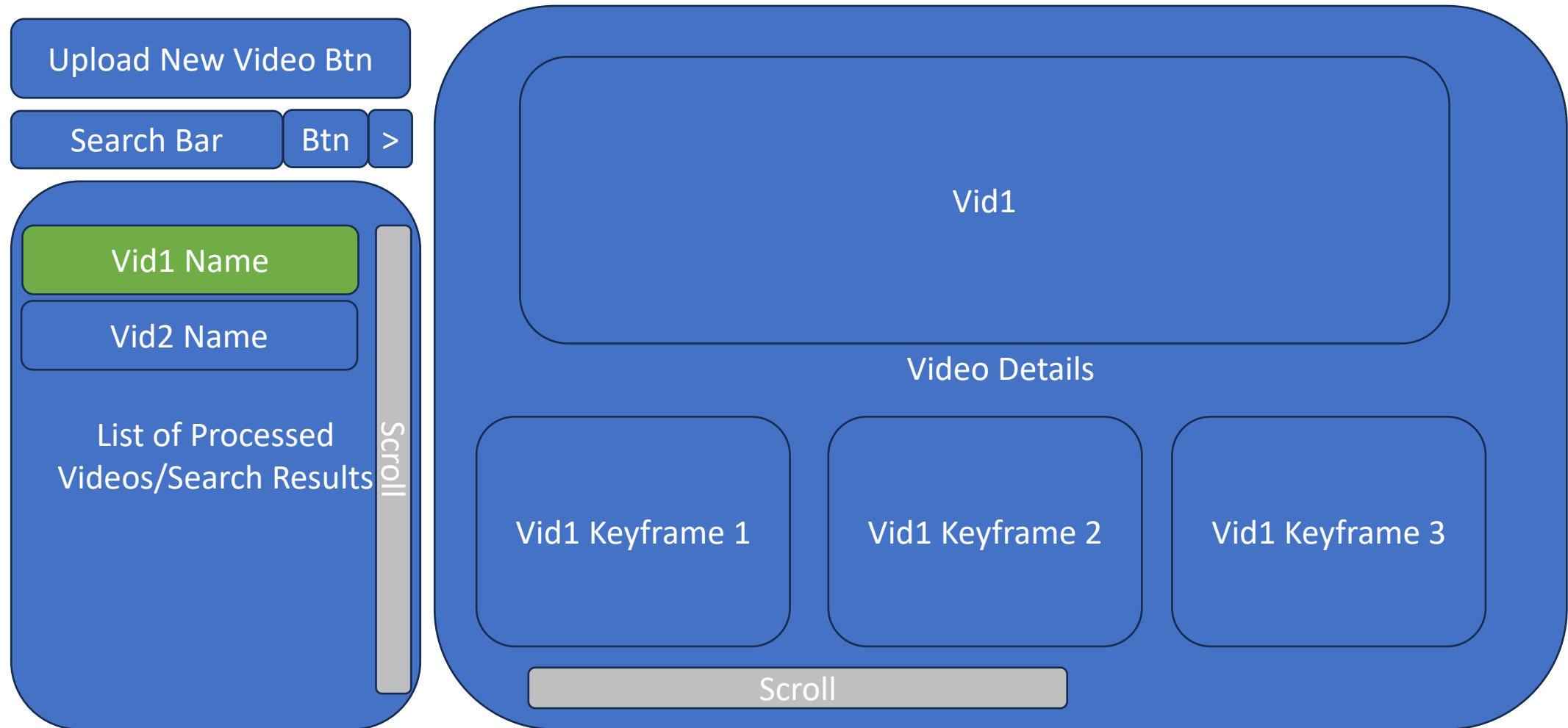


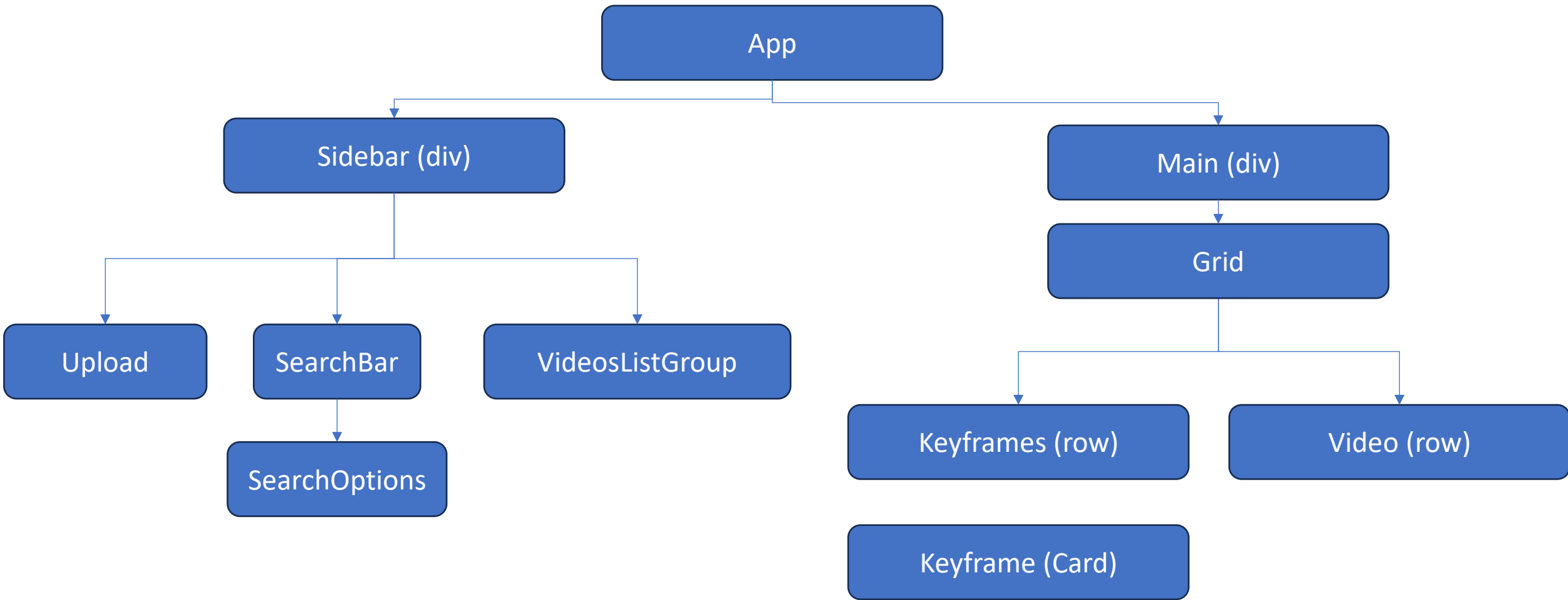
Architecture

Frontend

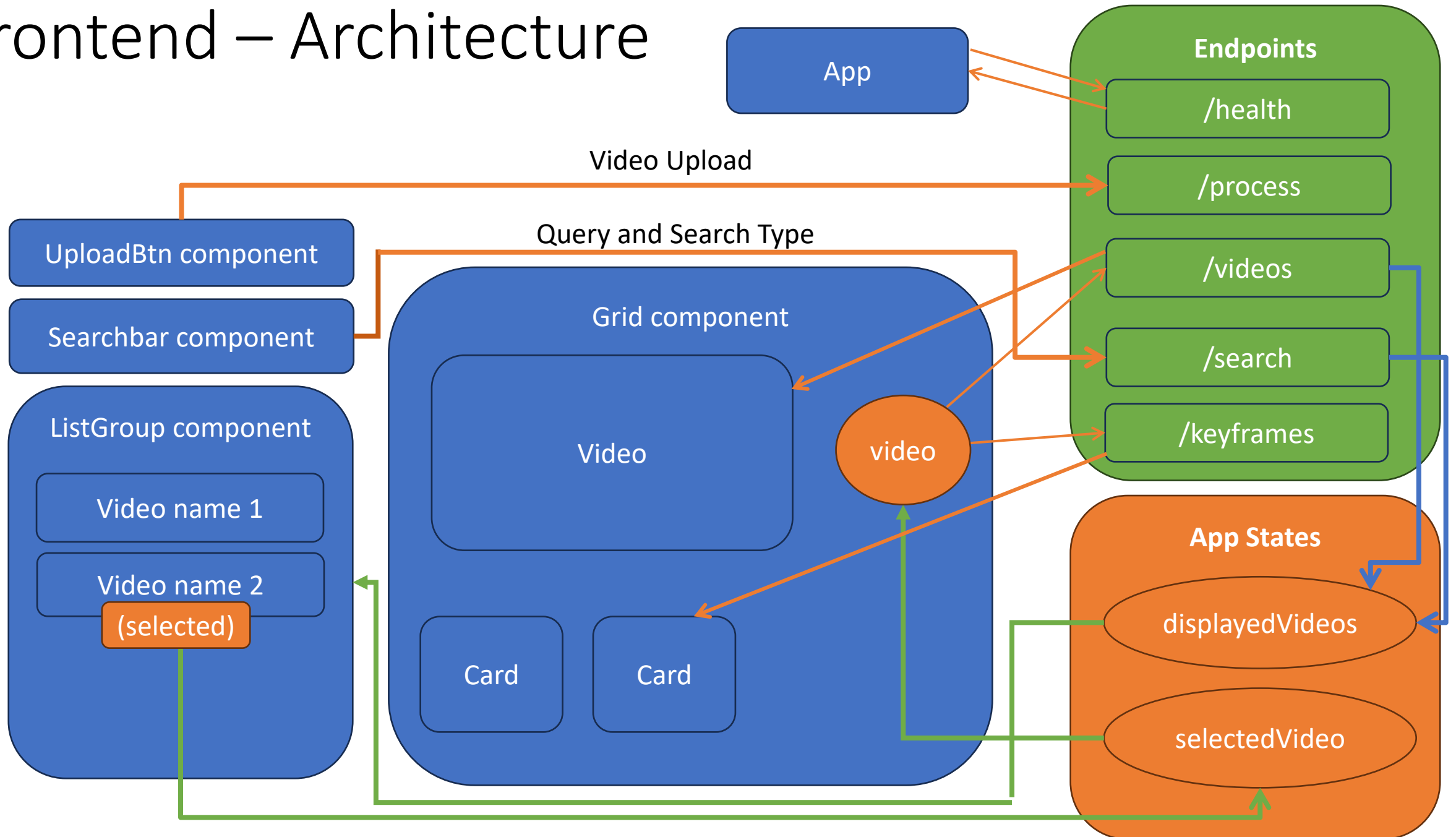
Frontend – Simple UI Layout



Frontend – Structure/DOM



Frontend – Architecture



Backend

Backend Architecture

Attempts to follow MVC structure

```
backend_architecture.txt
1  backend/
2  |  src/
3  |  |  services/
4  |  |  |  video_service.py    #Provides key frame extraction, object detection and writing to database functionality
5  |  |  |  search_service.py  #Provides search functionality
6  |  |  |  schemas/           #Schemas for sending model data in JSON
7  |  |  |  |  __init__.py
8  |  |  |  |  video_schema.py
9  |  |  |  models.py          #Models for database
10 |  |  |  routes.py           #Acts as the controller
11 |  |  tests/
12 |  |  |  test_suite.py       #tests for endpoints, object detection accuracy and frame extraction
13 |  |  keyframes/
14 |  |  |  video_name/
15 |  |  |  |  0.jpg            #keyframes saved and accessed by /video_name/int in keyframes folder
16 |  |  uploads/
17 |  |  |  video_name.mp4      #video files saved and accessed by video name in uploads folder
18 |  |  instance/
19 |  |  |  processed_videos.db  #SQLite .db file
20 |  |  templates/             #html files for backend testing
21 |  |  /env                    #dependencies
22 |  |  app.py                  #main entry point
23 |  |  requirements.txt        #for docker buildfile to build necessary dependencies
24 |  |  .dockerignore           #for docker build
25 |  |  Dockerfile              #for docker build
26 |  |  MobileNetSSD_deploy.caffemodel
27 |  |  MobileNetSSD_deploy.prototxt  #Object Detection models
28 |
```

Backend - endpoints

- 4 endpoints
 - GET /health: Return status of service
 - POST /process: Accepts video files, performs key frame extraction, object detection and saves results to database
 - GET /videos: Retrieves all processed videos from the database
 - GET /search: Performs full-text search on video summaries based on detected objects or video filename

Backend - /health GET

- Returns status of the service when endpoint is hit

Backend - /process POST

- Expects uploaded video file as an argument
- Saves file to /uploads folder
- Utilize custom MobileNetProcessor Class to process video
- Return HTTP code

Backend - /videos GET

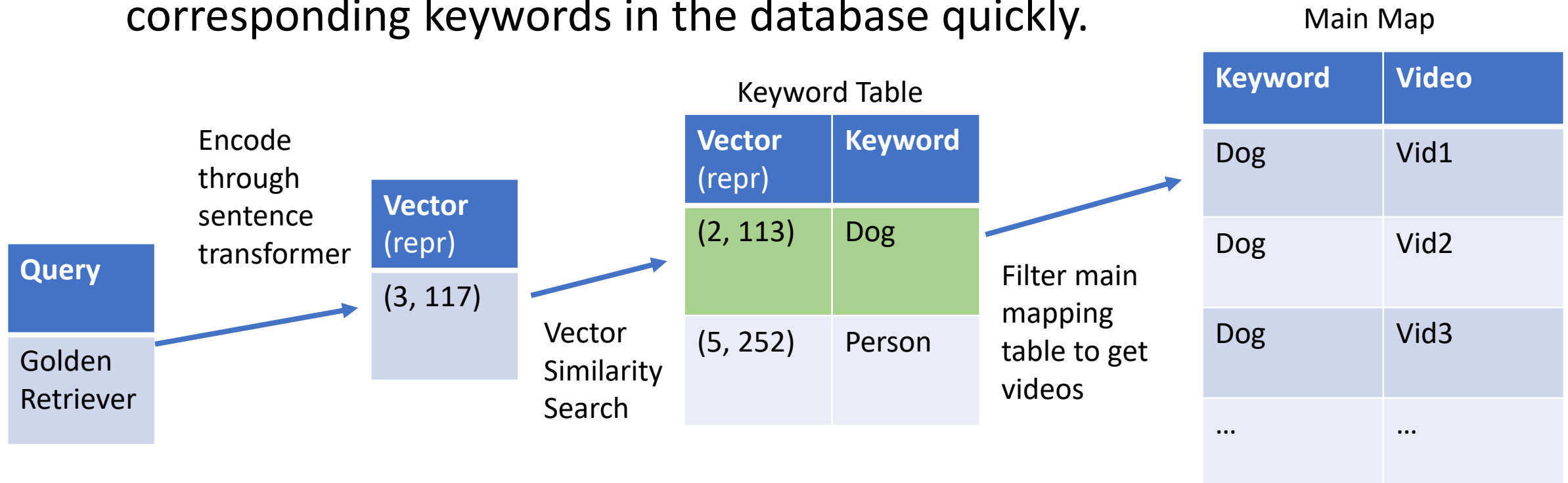
- Retrieves all videos from the video table in database
- Returns list of JSON objects to the requestor as video schema
- /videos/<int:id> will return the video file resource saved in uploads folder

Backend - /search GET

- Expects query and search_type as arguments
- Utilize custom SearchHandler Class to return list of videos from the video table in database
- Return list of JSON objects to the requestor as video schema
- If search is blank, return all videos

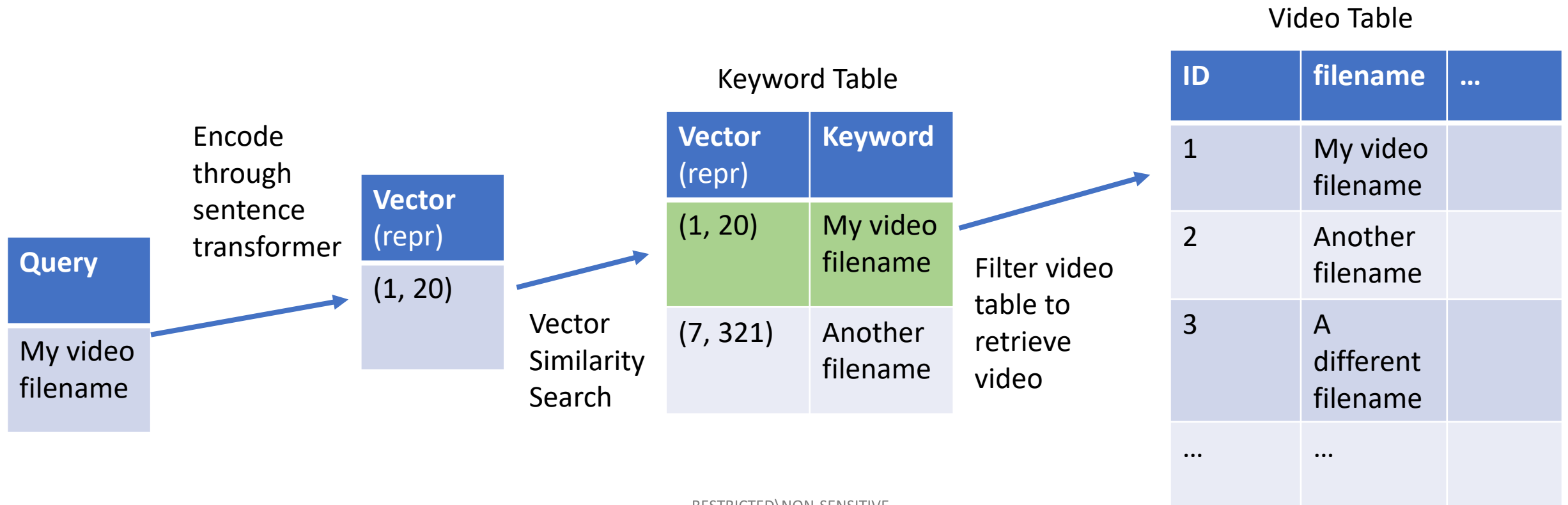
Database structure and VSS approach

- The database should include a “keyword” table of detected objects to their numerical vectors in embedded space. This way, the nearest neighbors from vector similarity search can be mapped back to their corresponding keywords in the database quickly.



Database structure and VSS approach

- For filename search, the search function instead looks at the Video table instead of the main map



Database structure and VSS approach

- KDTree approach from Scikit-learn is used for Vector Similarity Search
- For corpus size less than leaf-size, brute force knn approach is used
- When corpus size exceeds leaf-size, search function will travel along KDTree branches resulting in ann behaviour

Database structure and VSS approach

- For each video submitted, saves the following information to the SQLite database:
 - Video filename – retrieved from uploaded file
 - Detected objects – retrieved from `get_detected_objects()`
 - Frame timestamps – retrieved from `get_framets_keyword_dict()`
 - Created timestamp – retrieved at user input
- Detected objects have a relationship to frame timestamps within each video

Database structure and VSS approach

- Detected objects and filenames are vectorized and added to the keyword table (vectors map) as unique entries. Frame timestamps are ignored in vectors map since they're not useful as a search parameter. The filename is not added to the main map to avoid being confused as a detected object, as the main map will be used for detected objects search

Vectors Map

ID	Vector (rep)	Keyword	Created
1	(2, 113)	Dog	(older ts)
2	(5, 252)	Person	~
3	(51, 384)	TV	15:27:05 3 Jan
4	(93, 227)	filename	~
..	...	...	...

Video Processing Results

Dog	TV	0:37	filename	15:27:05 3 Jan
-----	----	------	----------	----------------

Entries are unique so ignore existing
Add new keywords and their embedded vectors

Add all mappings to main mapping table

Main Map

Flatten table as much as possible since most processing outputs have a many to many RS with videos

Vid id	Video (rep)	KW id	KW (rep)	F_ts	Created
4	Vid4	3	TV	0:0	15:27:05 3 Jan
4	Vid4	1	Dog	0:37	~
5	...	...	...	...	~

Database structure and VSS approach

Keyword Table
(aka vector map)

ID	Vector (rep)	Keyword	Created
1	(2, 113)	Dog	
2	(5, 252)	Person	
3	(51, 384)	TV	
4	(93, 227)	filename	
..	...	...	

Main Mapping Table

ID	Vid id	KW id	F_ts	Created at
1	4	1	0:0	15:27:05 3 Jan
2	4	2	0:0	15:27:05 3 Jan
3	4	3	0:37	
4	4	4	...	

Videos Table (duplicate entries for filenames are ok)

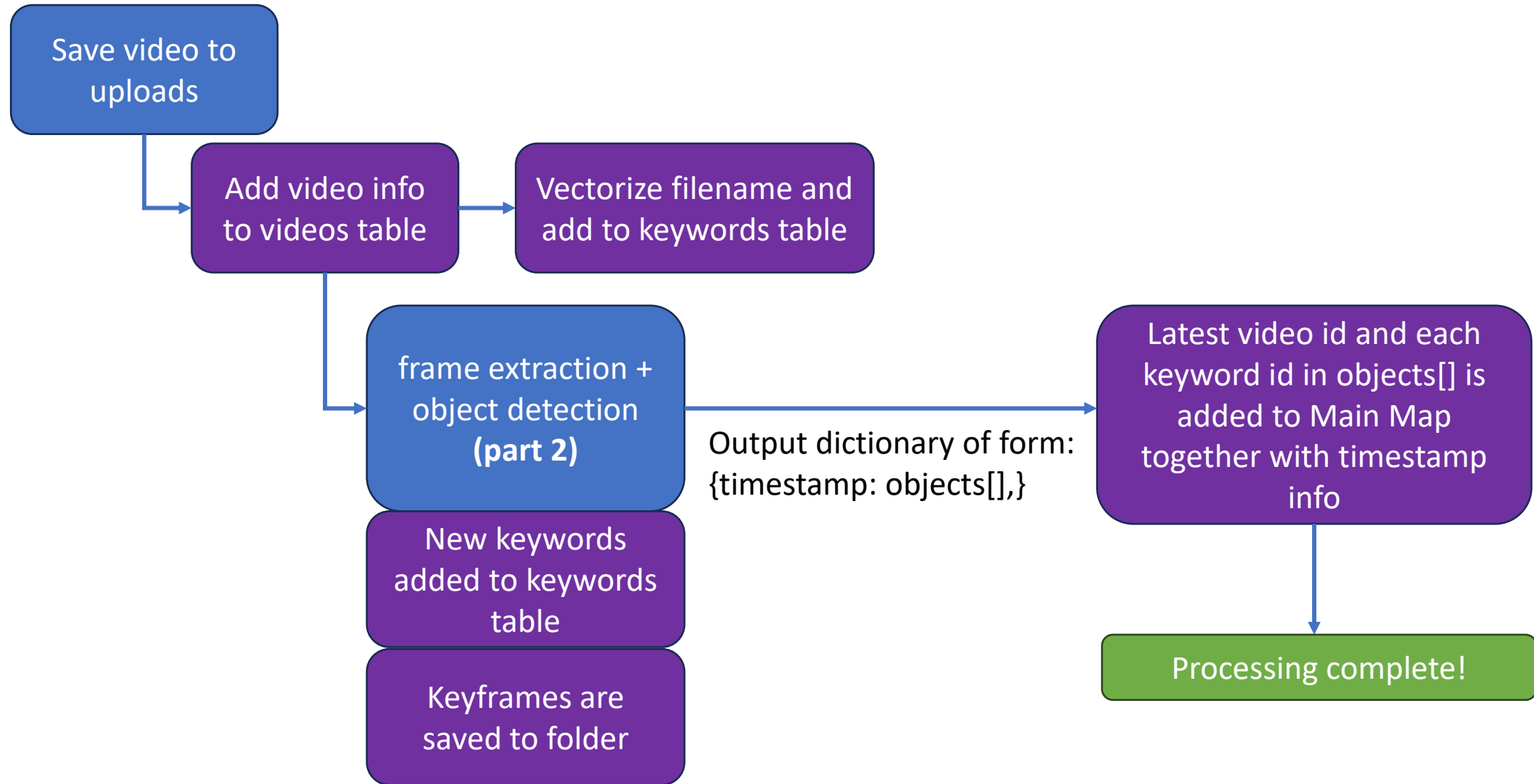
ID	Video filename	URI	Created
1	Vid1	/videos/1	HH:MM:SS
2	Vid2	/videos/2	~
3	Vid3	/videos/3	~
4	Vid4	/videos/4	~
..	...	...	...

Video Processing Pipeline

- Handled by MobileNetProcessor, child class of VideoProcessor
- Accepts file_path to a video as argument when initializing
- Contains methods to extract interesting frames, detect objects within each frame, and save details to the database

Video Processing Pipeline – logical flow part 1

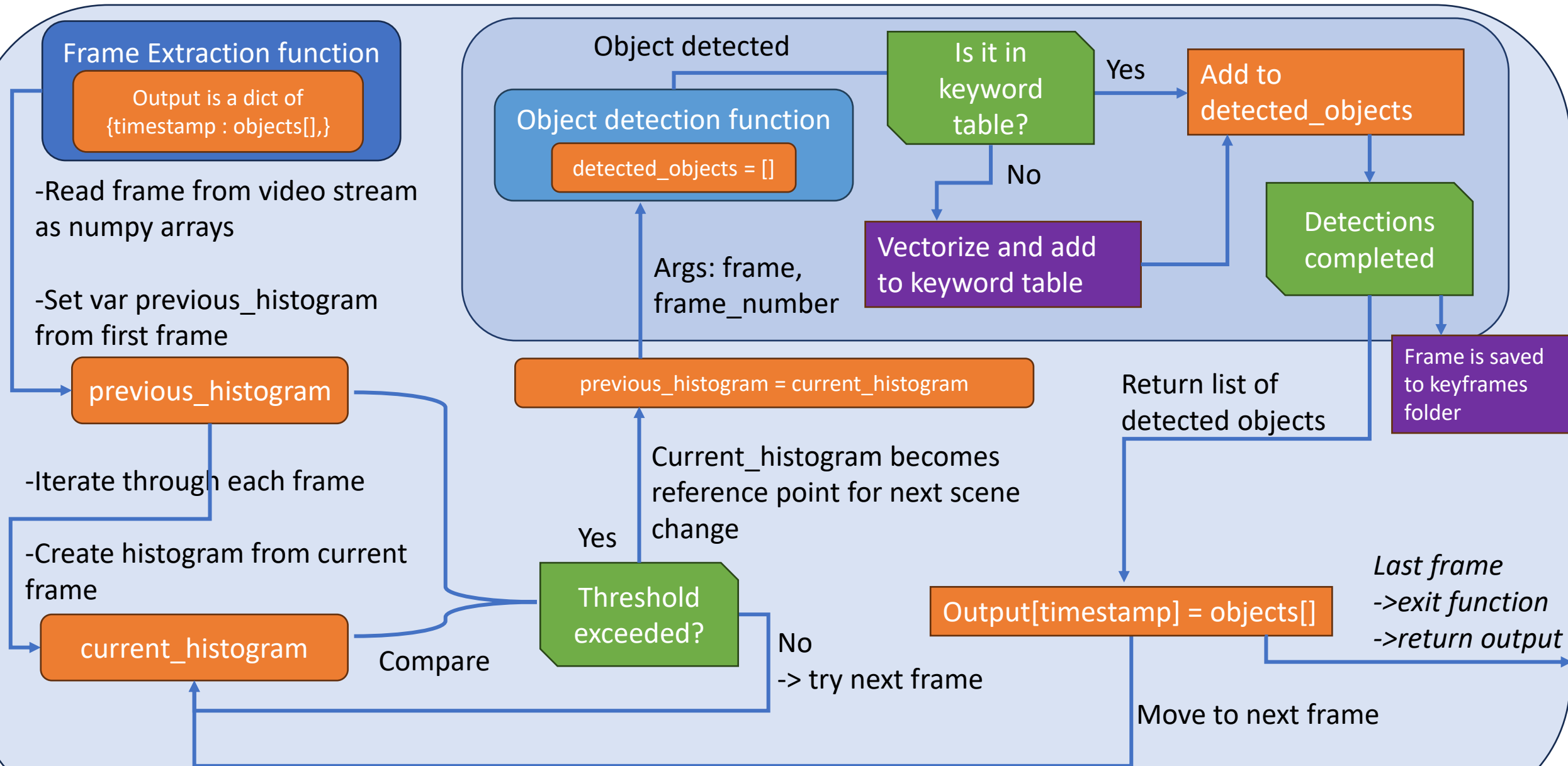
RESTRICTED\NON-SENSITIVE



RESTRICTED\NON-SENSITIVE

Video Processing Pipeline – logical flow pt1

RESTRICTED\NON-SENSITIVE



RESTRICTED\NON-SENSITIVE

Video Processing Pipeline – Key frame extraction

- MobileNetProcessor uses a histogram comparison method to detect scene changes
- Histograms represent the distribution of pixel color intensity for each channel, for each frame. Comparing the histograms of different frames allows us to detect changes when there is a significant difference in the lighting or color between the frames.
- However, there is no spatial information captured in the histograms. Since this is basically a pixel counting operation, the computational overhead is low, but we won't be able to detect some changes if they are too similar in color spread and brightness.
- This is especially true for videos which are darker, like night-time scenes

Video Processing Pipeline – Object Detection

- MobileNetProcessor relies on the MobileNetSSD model which is quite old but very lightweight. There is only a total of 21 classes.
- During testing with some videos, the accuracy observed was not the best. Amusingly, top-down footage of cars were identified as dining tables. Recognition of people and animals performed decently.
- MobileNetProcessor is a child class of VideoProcessor which acts as a base for future implementations that may use different models.
- An improvement to be made is to decouple the keyframe extraction and object detection for higher modularity. For the assessment, it just felt faster to write the code this way.

Frontend Docs

App.tsx

States:

- displayedVideos – Video[]
- selectedVideo – Video

-Passes displayedVideos into ListGroup under sidebar to show either all processed videos, or search results

-Passes selectedVideo into Grid component to display video and its details

UploadBtn.tsx

States:

- buttonDisabled : bool

-Takes in video file as form input. Submit button posts file to /process route on the backend

-buttonDisabled state checks if there is a file uploaded in the form and disables it otherwise to prevent posting null to the endpoint

Searchbar.tsx

States:

- query : string
- search_type : number

Props:

- option1 : string

Display name for first radio button

- option2 : string

Display name for second radio button

- handleResponse : (Video[]) => void

Function called on a list of Video data structs expected from /search after submitting search query with “query” and “search_type” parameters

-Takes in user input as a string and sends a query to the backend for a list of videos matching the query.

-User can toggle between search by file name or search by detected objects with the radio buttons

ListGroup.tsx

States:

- selectedIndex : number

Key to indicate selected list item for visual update and

Props:

- videos : Video[]

List of Video interfaces to display

- heading : string

Header text for ListGroup component

- onSelectItem : (Video) => void

Do something with Video item that is selected

- Displays list of all videos or search results
- Allows user to select video to see details of

Grid.tsx

States:

- keyframes

List of timestamps for extracted keyframes for the video

Props:

- children : string

Header text for Grid component

- video : Video

Video interface containing model data for the selected video

- Displays video, video title and extracted keyframes for a selected video, passed in as a prop from App
- Creates a row of Card components representing the extracted keyframes in a horizontal scrollable container

Card.tsx

Props:

- `image_uri : string`

Path pointing to folder with video name

- `frame_number : number`

Number pointing to the image file within specified folder

- `date_created : string`

Date the keyframe was saved

- `frame_timestamp : string`

Timestamp of the keyframe in the video

-Displays keyframe image with labels, frame timestamp, and date created